

بسم الله الرحمن الرحيم

تمرین سوم درس سیستم های توزیع شده

سید محمد جزایری - 810101399

- 2.....(بخش تئوری)
- 4.....(بخش عملی)
- 4.....۱. معماری سیستم ((System Architecture
- 4.....۲. راهنمای اجرای سیستم
- 4.....الف) نحوه اجرای رپلیکاها ((Replica Nodes
- 5.....ب) نحوه اجرای کلاینت آزمایش گر ((Test Client
- 5.....۳. مدل های سازگاری پیاده سازی شده ((Consistency Models
- 5.....الف) سازگاری نهایی ((Eventual Consistency
- 6.....ب) سازگاری قوی ساده شده ((Strong Consistency via Quorum
- 6.....۴. سیاست نسخه گذاری و حل تعارض ((Versioning & Conflict Resolution
- 7.....۵. نتایج و تحلیل دقیق سناریوهای چهارگانه
- 8.....سناریوی ۱: مشاهده ناسازگاری موقت ((Temporary Inconsistency
- 9.....سناریوی ۲: خرابی یک رپلیکا ((Replica Failure Experiments
- 9.....سناریوی ۳: تعارض همزمان ((Concurrent Writes Conflict Resolution
- 10.....سناریوی ۴: تاثیر تاخیر شبکه ((Network Latency Impact Benchmarks
- 11.....۶. جدول نمونه متریک های گزارش ((Metrics Matrix
- 13.....۷. تحلیل تفاوت ها (CAP Theorem) و محدودیت های پیاده سازی
- 13.....الف) تحلیل قضیه CAP
- 13.....ب) مشکلات و محدودیت های پیاده سازی

بخش تئوری

۱. چرا داده‌ها در سیستم‌های توزیع‌شده Replicate می‌شوند؟ تکثیر داده‌ها (Replication) دو دلیل عمده دارد: افزایش قابلیت اطمینان (Reliability/Availability) تا در صورت خرابی یک نود، داده‌ها از بین نروند و سیستم همچنان پاسخگو باشد؛ و افزایش کارایی (Performance) تا کلاینت‌ها بتوانند داده‌ها را از نزدیک‌ترین یا خلوت‌ترین نود (Replica) بخوانند که باعث کاهش تاخیر (Latency) می‌شود.

۲. تفاوت Replication برای افزایش کارایی و Replication برای تحمل خطا:

- **برای کارایی (Performance):** تمرکز بر کاهش زمان پاسخ‌گویی است. داده‌ها معمولاً در نزدیکی کاربر نهایی کپی می‌شوند (مثل CDNها) یا برای تقسیم بار (Load Balancing) روی چند سرور پخش می‌شوند.
- **برای تحمل خطا (Fault Tolerance):** تمرکز بر افزونگی (Redundancy) است. هدف این است که اگر یک یا چند Replica از کار افتادند (Crash)، سیستم بتواند با استفاده از کپی‌های دیگر به کار خود ادامه دهد و داده‌ای از دست نرود.

۳. مشکل اصلی Consistency از نظر Replication چیست؟ مشکل اصلی «هماهنگ نگه‌داشتن» نسخه‌های مختلف داده است. وقتی داده‌ای در یک Replica به‌روزرسانی می‌شود، مدتی طول می‌کشد تا این تغییر به سایر Replicaها منتقل شود. در این فاصله زمانی، کلاینت‌هایی که از سایر Replicaها می‌خوانند، داده‌های قدیمی (Stale Data) را مشاهده می‌کنند که باعث ناسازگاری در سیستم می‌شود.

۴. تفاوت Strong Consistency و Eventual Consistency:

- **Strong Consistency**: سیستم تضمین می‌کند که پس از پایان یک عملیات نوشتن، هر عملیات خواندنی (از هر Replica) قطعاً جدیدترین مقدار نوشته‌شده را برمی‌گرداند. این مدل نیازمند هماهنگی بالاست و باعث تاخیر می‌شود.

- **Eventual Consistency**: سیستم تضمین نمی‌کند که بلافاصله پس از نوشتن، همه Replicaها مقدار جدید را داشته باشند، اما تضمین می‌کند که اگر نوشتن جدیدی رخ ندهد، "در نهایت" (Eventually) همه Replicaها به یک وضعیت مشترک و یکسان همگرا (Converge) خواهند شد.

۵. مفهوم Read-your-writes Consistency: این یک مدل سازگاری کلاینت‌محور (Client-centric) است. تضمین می‌کند که اگر یک کاربر (کلاینت) مقداری را در سیستم بنویسد، در درخواست‌های خواندن بعدی خودش، حتماً همان مقدار جدید (یا مقداری جدیدتر از آن) را خواهد دید و هرگز مقدار قدیمی‌تر از نوشته خودش به او برگردانده نمی‌شود.

۶. مفهوم Monotonic Reads: تضمین می‌کند که اگر یک کلاینت مقدار داده‌ای را در زمان t_1 بخواند، هرگز در خواندن‌های بعدی (زمان $t_2 > t_1$) مقداری قدیمی‌تر از آنچه در t_1 دیده است را نخواهد خواند. به عبارتی، زمان برای کلاینت هرگز به عقب برنمی‌گردد.

۷. چرا نمی‌توان همزمان Availability، Consistency و Partition Tolerance را کامل داشت؟ (قضیه CAP) قضیه CAP بیان می‌کند که در صورت بروز قطعی شبکه (Partition)، سیستم توزیع‌شده باید بین Consistency (پاسخ قطعی و صحیح دادن یا خطا دادن) و Availability (همیشه پاسخ دادن حتی با داده‌های قدیمی) یکی را انتخاب کند. اگر ارتباط دو Replica قطع شود، و شما بخواهید سیستم Available بماند، مجاز به ثبت داده‌های جدید هستید اما این داده‌ها نمی‌توانند به Replica قطع‌شده برسند، پس Consistency نقض می‌شود. اگر بخواهید Consistency حفظ شود، باید تا زمان وصل شدن شبکه از پذیرش درخواست جدید خودداری کنید، که Availability نقض می‌شود.

بخش عملی)

۱. معماری سیستم (System Architecture)

معماری این پروژه بر پایه یک خوشه (Cluster) توزیع شده شامل ۳ گره رپلیکای مستقل (Replica Node) طراحی شده است. هر رپلیکا به عنوان یک پردازش (Process) کاملاً مجزا در سیستم عامل اجرا می شود و از طریق پروتکل HTTP شبکه با کلاینت و سایر گره ها ارتباط برقرار می کند.

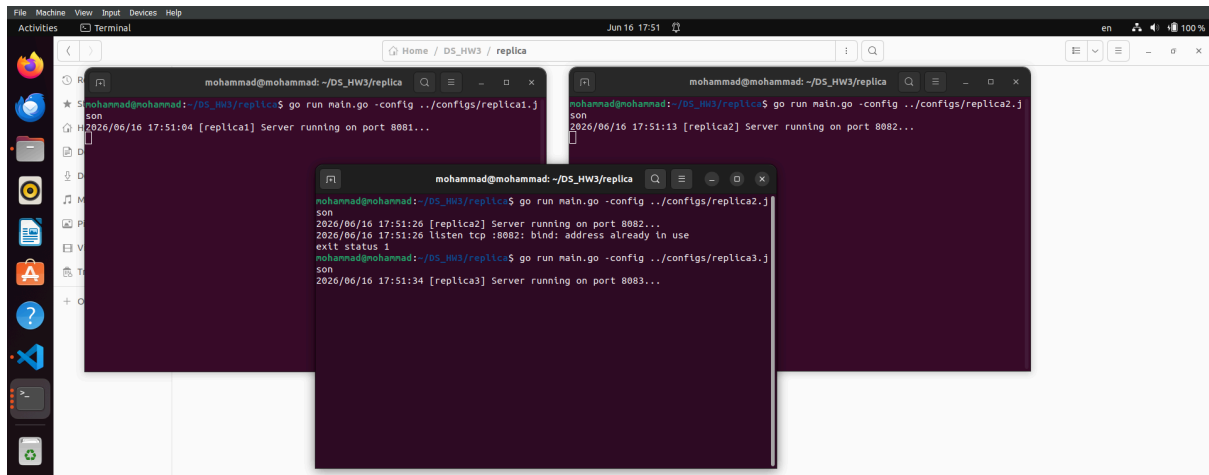
- **لایه ذخیره سازی:** داده ها در هر گره به صورت محلی و در حافظه (In-Memory) درون یک نگاشت (map) نگهداری می شوند. جهت تضمین ایمنی دسترسی همزمان (Thread-Safety) در برابر درخواست های موازی، از مکانیزم قفل خواندن/نوشتن (sync.RWMutex) استفاده شده است.
- **لایه ارتباطی:** تعاملات گره ها با کلاینت و با یکدیگر از طریق بسته های متنی JSON بر بستر API های سرور HTTP ساده پیاده سازی شده است.

۲. راهنمای اجرای سیستم

ساختار دیرکتوری پروژه کاملاً منطبق بر الگوی پیشنهادی صورت تمرین است. برای اجرای ارزیابی، مراحل زیر طی می شود:

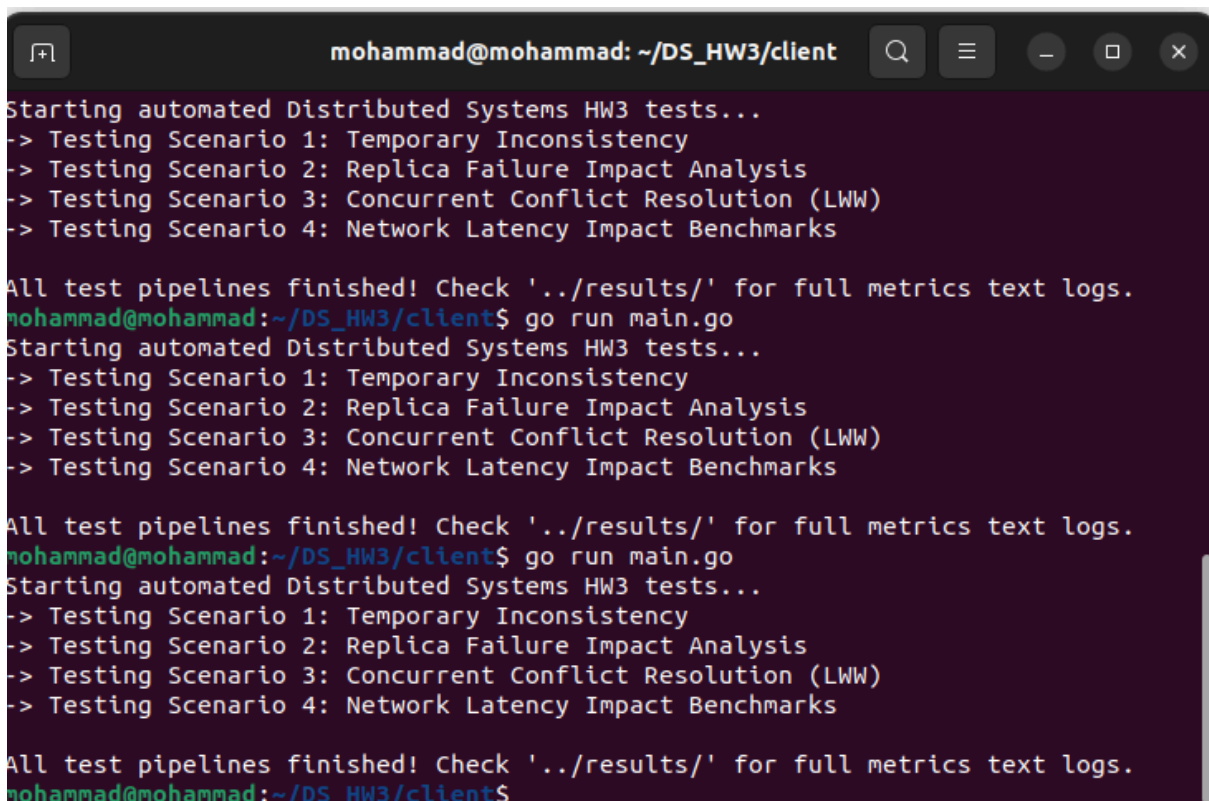
الف) نحوه اجرای رپلیکاها (Replica Nodes)

سه ترمینال مجزا باز کرده و هر گره را با فایل پیکربندی اختصاصی خود مطابق تصویر زیر اجرا می کنیم (در README جزئیات بیشتری ذکر شده اند).



(ب) نحوه اجرای کلاینت آزمایش‌گر (Test Client)

یک ترمینال جدید در پوشه client / باز کرده و مطابق تصویر پایین چهار سناریو را اجرا می‌کنیم.



۳. مدل‌های سازگاری پیاده‌سازی شده (Consistency Models)

الف) سازگاری نهایی (Eventual Consistency)

در این مدل، پس از دریافت درخواست PUT توسط گره هماهنگ‌کننده (Coordinator)، داده فوراً روی حافظه محلی اعمال شده و پاسخ موفقیت به کلاینت بازگردانده می‌شود. عملیات تکثیر داده روی سایر رپلیکاها به صورت کاملاً ناهمگام (Asynchronous) و از طریق Go Goroutines در پس‌زمینه شبکه هدایت می‌شود.

ب) سازگاری قوی ساده‌شده (Strong Consistency via Quorum)

در این مدل، عملیات PUT زمانی موفقیت‌آمیز تلقی می‌شود که داده جدید توسط اکثریت گره‌های شبکه (Quorum) تایید و ثبت شده باشد. فرمول حد نصاب سیستم برای تعداد $N = 3$ رپلیکا به شرح زیر است:

$$Quorum(W) = \text{floor}(\frac{N}{2}) + 1 \rightarrow \text{floor}(\frac{3}{2}) + 1 = 1 + 1 = 2$$

بنابراین، گره هماهنگ‌کننده تا زمان دریافت تاییدیه (Ack) از حداقل ۲ رپلیکا (مثلاً خودش و یک گره همتا) مسدود (Block) می‌ماند و سپس پاسخ را به کلاینت می‌دهد.

۴. سیاست نسخه‌گذاری و حل تعارض (Versioning & Conflict Resolution)

- ساختار داده‌ای متادیتا: برای هر کلید، ساختار داده‌ای شامل مقدار کلید، خود داده، شماره نسخه، و شناسه آخرین گره به‌روزرسانی‌کننده ذخیره می‌شود:

```
type DataItem struct {
    Key      string `json:"key"`
    Value     string `json:"value"`
    Version  int    `json:"version"`
    UpdatedBy string `json:"updated_by"`
}
```

- سیاست نسخه‌گذاری (Versioning): با هر بار تغییر موفقیت‌آمیز داده، مقدار عددی version یک واحد افزایش می‌یابد. اگر گره‌ای داده‌ای با نسخه پایین‌تر از نسخه فعلی خود دریافت کند، برای جلوگیری از بازنویسی داده‌های جدید توسط بسته‌های تاخیر یافته شبکه، درخواست را نادیده می‌گیرد.

- **حل تعارض (Conflict Resolution):** از استراتژی (Last-Write-Wins (LWW)

استفاده شده است. در صورت وجود تلاقی همزمان (نسخه‌های کاملاً یکسان اما مقادیر متفاوت)، سیستم از شناسه گره‌ها (updated_by) به عنوان Tie-breaker استفاده می‌کند. هر گره‌ای که نام رشته‌ای بزرگتری داشته باشد (مثلاً "replica2" بزرگتر از "replica1")، داده‌ی آن به عنوان مقدار پیروز مستقر می‌شود.

۵. نتایج و تحلیل دقیق سناریوهای چهارگانه

برای اینکه بتوانیم با یک برنامه (client) تمام سناریوها را اجرا کنیم از موارد زیر کمک گرفتیم، ساختار داده اول توسط کلاینت پر می شود مقادیر key و value که مطابق صورت مسئله طراحی شده‌اند. اما consistency مشخص می کند که از کدام مدل (strong یا eventual) داریم استفاده می کنیم و DelayMs هم مشخص می کند که این درخواست با چه تاخیری (به میلی ثانیه) باید ارسال شود. ساختار داده دوم هم توسط replica پر می شود و message موفقیت آمیز بودن یا نبودن و دلیل شکست را به کلاینت می گوید، UpdatedReplicas تعداد replica های بروز شده را برمی‌گرداند و Version هم مطابق صورت مسئله طراحی شده‌اند.

```
type PutRequest struct {
    Key      string `json:"key"`
    Value    string `json:"value"`
    Consistency string `json:"consistency"`
    DelayMs  int    `json:"delay_ms"`
}

type PutResponse struct {
    Message      string `json:"message"`
    UpdatedReplicas int    `json:"updated_replicas"`
    Version      int    `json:"version"`
}
```

برای اینکه هر replica به درستی اجرا شود و اطلاعات خود را بداند به متغیرهای پایین نیاز دارد

```

type Config struct {
    ID      string `json:"id"`
    Port    string `json:"port"`
    Peers   []string `json:"peers"`
}

var (
    config Config
    store  = make(map[string]DataItem)
    mu     sync.RWMutex
)

```

با اطلاعات بالا هر replica آیدی خود، پورت خود و آدرس شبکه دیگر گره های موجود را خواهد فهمید. و با متغیر پایین تر این کانفیگ را ذخیره می کند و یک hash table می سازد و برای قفل کردن و جلوگیری از نوشتن های همزمان روی hash table از مکانیزم mutex استفاده می کند. همچنین فایل کانفیگ در ابتدای اجرا و در آرگومان به replica پاس داده می شود.

سناریوی ۱: مشاهده ناسازگاری موقت (Temporary Inconsistency)

- **داده های ثبت شده:** زمان پاسخ نوشتن محلی (ms) 7.46، تعداد رپلیکای فوراً به روزرسانی شده برابر با ۱ نود، و نتیجه اولین خواندن از رپلیکا ۲ به عنوان Stale Read Detected (داده جدید هنوز موجود نیست) ثبت شد. پس از اتمام پنجره ناسازگاری، مقدار با موفقیت به 100 همگرا شد.
- **تحلیل علمی:** این آزمایش به طور تجربی وجود **Inconsistency Window** (پنجره ناسازگاری) را اثبات می کند. از آنجا که تکثیر در مدل Eventual ناهمگام است، کلاینت در فواصل زمانی اولیه با پدیده خواندن داده منقضی یا ناموجود مواجه می شود تا زمانی که همگرایی کامل رخ دهد.

scenario1.txt

```
1 === SCENARIO 1: TEMPORARY INCONSISTENCY LOGS ===
2 PUT Latency: 7.463388ms
3 Replicas Updated Immediately (Ack): 1
4 Immediate GET Latency from Replica 2: 2.350998ms
5 Immediate GET status: Stale Read Detected (Data not present yet)
6 Post-convergence GET status: Value is now '100' (Successfully Converged)
7 Total Stale Reads counted: 1
```

سناریوی ۲: خرابی یک رپلیکا (Replica Failure Experiments)

- داده‌های ثبت‌شده (در وضعیت قطع بودن Replica 3):

- مدل Eventual Consistency: زمان پاسخ (μs) 665 | تعداد نود آپدیت شده در زمان پاسخ = ۱.

- مدل Strong Consistency: زمان پاسخ (ms) 1.39 | تعداد نود آپدیت شده در زمان پاسخ = ۲ (کد پاسخ ۲۰۰ - موفق).

- **تحلیل علمی:** این آزمایش تحمل خطای سیستم را می‌سنجد. در مدل Eventual، خاموش بودن نودها تاثیری در Latency کلاینت ندارد چون نوشتن محلی است. در مدل Strong، با وجود قطع بودن کامل رپلیکا ۳، عملیات همچنان موفق است؛ زیرا حد نصاب اکثریت ($Quorum = 2$) توسط رپلیکاهای ۱ و ۲ احراز شده است. افزایش زمان پاسخ مدل (ms) 1.39 به دلیل تلاش ناموفق لایه شبکه برای برقراری ارتباط با گره معیوب ۳ قبل از حد نصاب است.

سناریوی ۳: تعارض همزمان (Concurrent Writes Conflict Resolution)

- داده‌های ثبت‌شده: پس از ارسال همزمان دو داده متفاوت به دو گره فعال، وضعیت نهایی همگرایی در هر ۳ گره به مقدار واحد زیر رسید:

scenario3.txt

```
1 === SCENARIO 3: CONCURRENT WRITES CONFLICT RESOLUTION ===
2 Replica Node [:8081] Converged State: Value=Value_From_Replica_2, Version=1, UpdatedBy=replica2
3 Replica Node [:8082] Converged State: Value=Value_From_Replica_2, Version=1, UpdatedBy=replica2
4 Replica Node [:8083] Converged State: Value=Value_From_Replica_2, Version=1, UpdatedBy=replica2
```

```

fmt.Fprintln(file, "=== SCENARIO 3: CONCURRENT WRITES CONFLICT RESOLUTION ===")
key := "conflict_key"

var wg sync.WaitGroup
wg.Add(2)

go func() {
    defer wg.Done()
    req, _ := json.Marshal(PutRequest{Key: key, Value: "Value_From_Replica_1", Consistency: "eventual", DelayMs: 0})
    _, _ = http.Post("http://localhost:8081/put", "application/json", bytes.NewBuffer(req))
}()

go func() {
    defer wg.Done()
    req, _ := json.Marshal(PutRequest{Key: key, Value: "Value_From_Replica_2", Consistency: "eventual", DelayMs: 0})
    _, _ = http.Post("http://localhost:8082/put", "application/json", bytes.NewBuffer(req))
}()

wg.Wait()
time.Sleep(1500 * time.Millisecond)

```

در شکل فوق نحوه ارسال همزمان به دو replica مختلف را مشاهده می کنیم.

- **تحلیل علمی:** این سناریو صحت الگوریتم حل تعارض LWW را تایید می کند. چون هر دو درخواست نسخه یکسان داشتند، مقایسه کاراکتری رشته شناسه گره ها اعمال شد. از آنجا که حروف الفبای "replica2" بزرگتر از "replica1" ارزیابی می شود، سیستم به صورت کاملاً قطعی داده گره دوم را پیروز تعارض اعلام کرده و شبکه به یکپارچگی ساختاری رسید.

سناریوی ۴: تاثیر تاخیر شبکه (Network Latency Impact Benchmarks)

- **تحلیل روند آماری:** بر اساس داده های ماتریسی، در مدل **Eventual** زمان پاسخ کلاینت (PUT Latency) همواره در کمترین حد ممکن (حدود 0.5 (ms) تا 1.2 (ms)) ثابت می ماند و تحت تاثیر تاخیر تزریق شده شبکه قرار نمی گیرد. با این حال، افزایش تاخیر شبکه مستقیماً پنجره ناسازگاری را گشادتر کرده و تعداد خواندن های Stale کلاینت را به شدت از ۱ خواندن (در تاخیر 0ms) به **۹۲ خواندن منقضی** (در تاخیر 2000ms) جهش می دهد.
- در مدل **Strong**، تعداد خواندن های Stale در تمام سطوح تاخیر دقیقاً 0 باقی می ماند که نشان از حفظ یکپارچگی خطی (Linearizability) دارد. اما هزینه این سازگاری بالا، افت شدید کارایی کلاینت است؛ به طوری که زمان پاسخ PUT به صورت خطی متناسب با تاخیر شبکه افزایش یافته و در تاخیر 2000 (ms) به عدد سنگین 2.003 (sec) می رسد.

۶. جدول نمونه متریک‌های گزارش (Metrics Matrix)

این جدول مطابق با قالب دقیق خواسته شده در بخش ۱۲ صورت تمرین تکمیل شده است:

Scenario	Model	PUT Latency	Convergence Time	Stale Reads Checked
1	Eventual	7.46 (ms)	46.927 (ms) با استناد به سطر چهارم	1
2	Eventual	1.42 (ms)	46.927 (ms) با استناد به سطر چهارم	1
2	Strong	8.299 (ms)	8.299 (ms) توضیحات در پایین جدول	0
4	Eventual (0 (ms))	499.555 (ms)	46.927 (ms)	1
4	Eventual (	523.418 (μs)	523.357 (ms)	22

	500 (ms))			
4	Eventual (2000 (ms))	1.205 (ms)	2011.991 (ms)	92
4	Strong (0 (ms))	1.307 (ms)	2.849 (ms)	0
4	Strong (500 (ms))	502.94 (ms)	504.292 (ms)	0
4	Strong (2000 (ms))	2003.958 (ms)	2005.079 (ms)	0

توضیحات: در مدل Strong Consistency، عملیات نوشتن از مکانیزم همگام (Synchronous Quorum) استفاده می‌کند. یعنی کلاینت منتظر می‌ماند (Block می‌شود) تا هماهنگ‌کننده مطمئن شود داده روی اکثریت نودها (۲ نود از ۳ نود) کاملاً نوشته شده است و سپس پاسخ موفقیت (Response Code=200) را پس می‌دهد. زمانی که کلاینت پاسخ را دریافت می‌کند، داده هم‌اکنون روی تمام نودهای زنده و در دسترس (رپلیکا ۱ و ۲) ثبت شده و سیستم در همان لحظه به همگرایی

کامل رسیده است. بنابراین در سازگاری قوی، زمان همگرایی همیشه دقیقاً برابر با زمان پاسخ نوشتن (PUT Latency) است، زیرا هیچ فرآیند پس‌زمینه یا پنجره ناسازگاری وجود ندارد که بعداً بخواهد همگرا شود. در خروجی ما تاخیر این بخش 8.299ms بود، پس زمان همگرایی نیز دقیقاً همین مقدار است.

۷. تحلیل تفاوت‌ها (CAP Theorem) و محدودیت‌های پیاده‌سازی

الف) تحلیل قضیه CAP

نتایج عددی بالا تجسم عینی **قضیه CAP** در سیستم‌های توزیع‌شده است.

- مدل **Strong Consistency** پایداری داده را فدای کارایی و دسترس‌پذیری می‌کند (انتخاب مرز CP). این مدل تضمین می‌کند داده قدیمی خوانده نشود اما با قطع نودها یا کندی شبکه، کل سیستم کند یا متوقف می‌شود.
- مدل **Eventual Consistency** دسترس‌پذیری و سرعت بالا را انتخاب می‌کند (انتخاب مرز AP). سیستم بسیار سریع پاسخ می‌دهد اما پذیرای ناسازگاری موقت و خواندن‌های Stale در طول شبکه خواهد بود.

ب) مشکلات و محدودیت‌های پیاده‌سازی

۱. وابستگی الگوریتم LWW به شناسه گره در زمان موازی‌بودن دقیق: اگر دو گره در یک زمان دقیق داده‌ای با نسخه یکسان ثبت کنند، متد شکننده تعارض همیشه به نفع نودی با نام بزرگتر (مانند رپلیکا ۲) رای می‌دهد که این امر می‌تواند سوگیری (Bias) در پذیرش داده‌ها ایجاد کند.
۲. پایداری داده حافظه (In-Memory Isolation): از آنجا که ذخیره‌سازی روی دیسک سخت صورت نمی‌گیرد، در صورت کرش کردن همزمان پردازش‌ها، تمامی تاریخچه‌ها و نسخه‌های ثبت شده از بین خواهند رفت.